# 2. Espacio de trabajo y ventanas

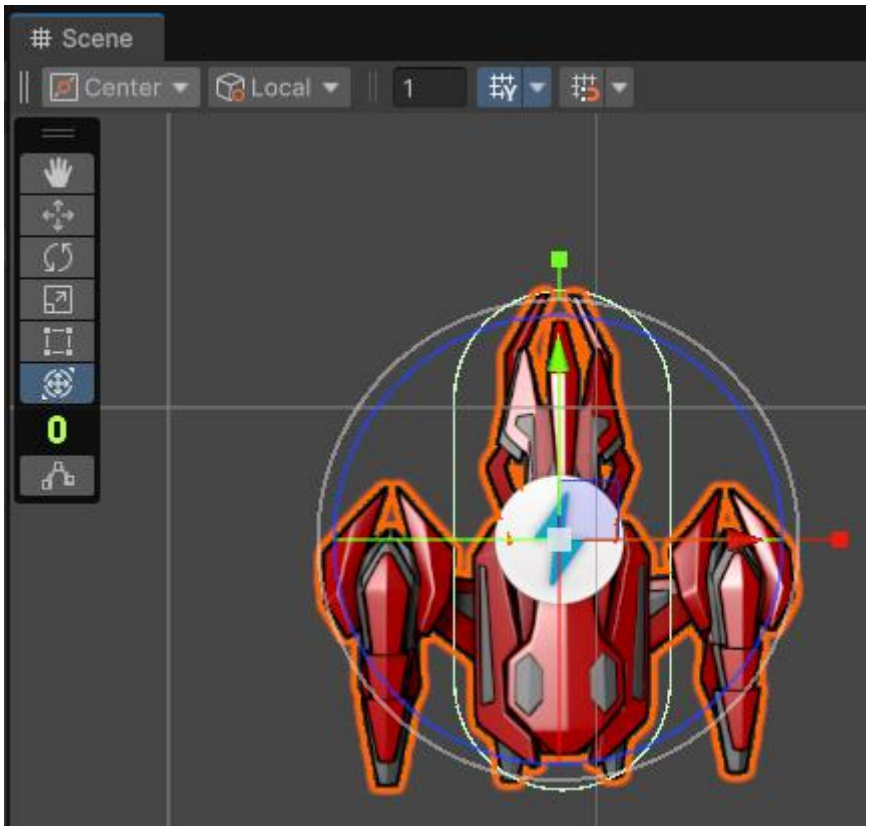
Unity organiza su interfaz en un conjunto de ventanas que pueden moverse, acoplarse y personalizarse para adaptarse al flujo de trabajo del desarrollador. Cada ventana cumple un propósito concreto dentro del proceso de creación de un videojuego. Esta ficha resume las ventanas más relevantes y cómo se utilizan en un proyecto.

### 2.1. Scene View

La **Scene View** es el espacio donde se construye el nivel.
Permite:

- Seleccionar GameObjects.
- Mover, rotar y escalar elementos.
- Editar luces, cámaras, colisiones y áreas de navegación.
- Visualizar gizmos (ayudas visuales para scripts y componentes).

La barra de herramientas superior incluye las herramientas de transformación (Move, Rotate, Scale, Rect, etc.).

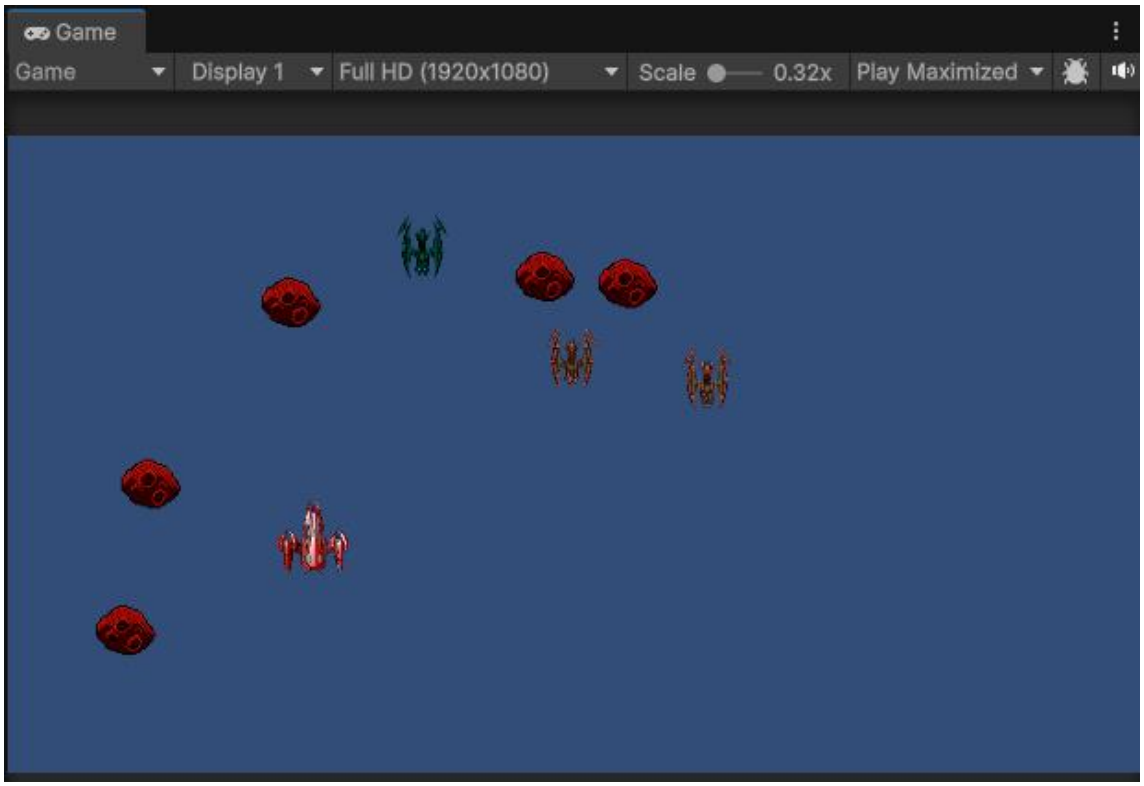


### 2.2. Game View

La **Game View** muestra el juego tal y como lo verá el jugador. Aspectos configurables:

- Modo de visualización (Free Aspect, resoluciones específicas…). Durante el desarrollo, para poder trabajar en una resolución fija, no dependiente del tamaño de la ventana, fijaremos Full HD, que es una proporción cómoda y muy frecuente de trabajo.
- Escala de visualización.
- Botón Stats para ver FPS, cantidad de draw calls, uso de memoria…
- Gizmos activables o no dentro del modo juego.

La Game View es especialmente útil cuando se prueban sistemas de movimiento, físicas, animaciones o HUD, como ocurre en scripts como **Jugador**, **Humano** o **Enemigo**.



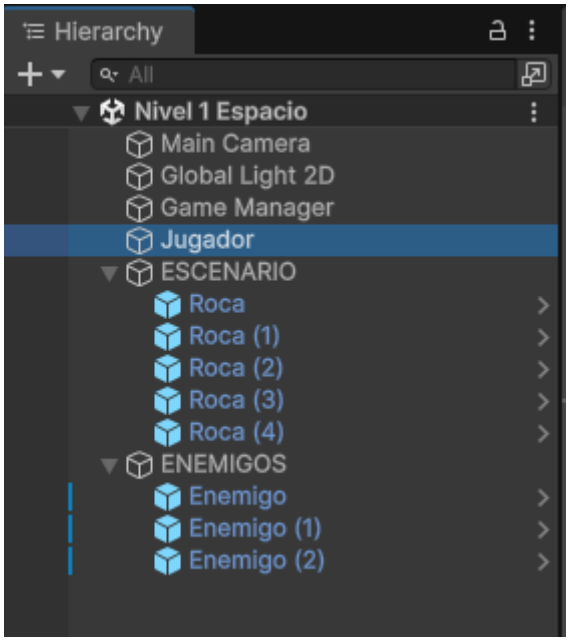
### 2.3. Hierarchy

La **Hierarchy** lista todos los GameObjects activos en la escena.
Las relaciones padre–hijo permiten agrupar elementos y organizarlos:

- Un personaje puede agrupar sprite, punto de disparo, collider y scripts.
- Un enemigo puede agrupar sus colisiones, efectos y puntos de disparo.
- La UI suele organizarse en Paneles dentro del Canvas.

Ejemplo típico relacionado con el script **Humano**:

- Humano (GameObject principal)
    - PuntoDisparo
    - Sprite
    - Collider2D
    - Animator




### 2.4. Inspector

El **Inspector** es una herramienta fundamental: permite ver y modificar todos los componentes de un GameObject seleccionado.

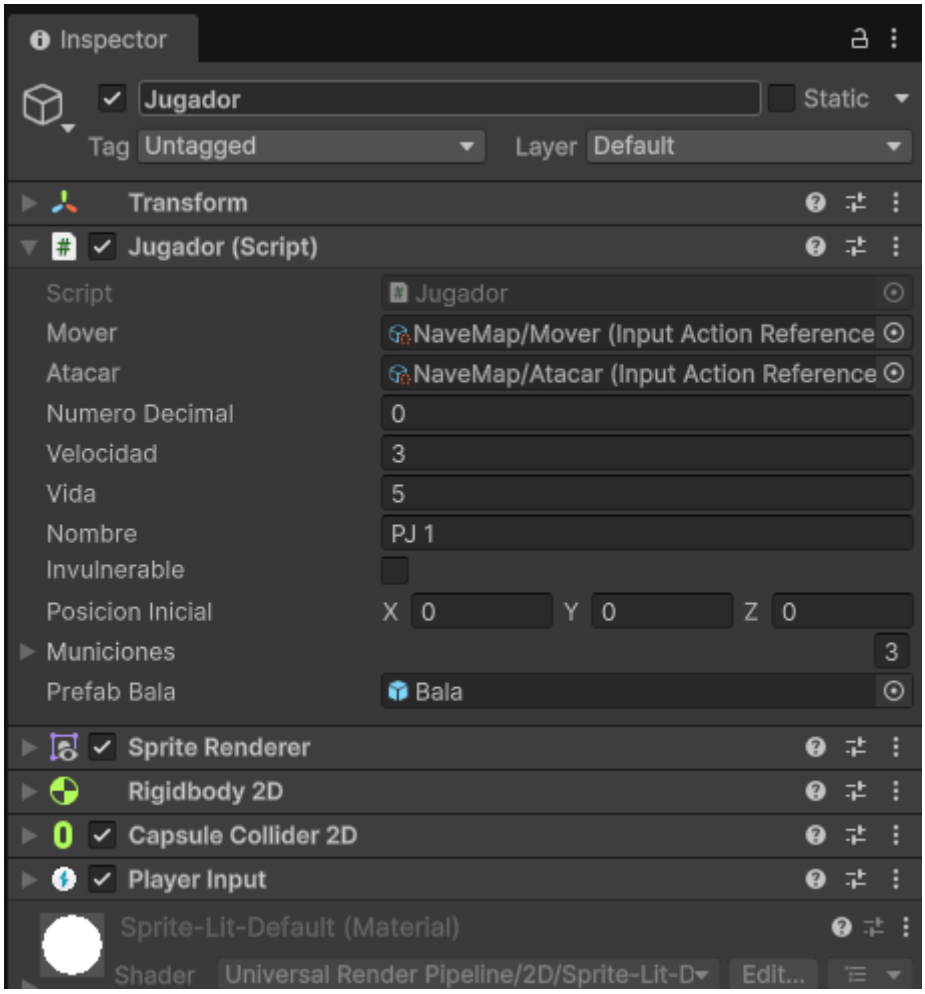
Desde el Inspector se pueden:

- Ajustar valores SerializeField de scripts:

```
[SerializeField] float velocidad = 5;
[SerializeField] GameObject prefabBala;
```

- Arrastrar referencias a objetos (prefabs, sprites, transforms).
- Modificar propiedades del Transform.
- Asignar sprites, físicas, materiales, fuentes, audio, etc.
- Activar/desactivar componentes.

El Inspector es esencial para configurar scripts como **Jugador**, **Enemigo**, **Humano**, **GameManager**, etc.




### 2.5. Project Window

El **Project** contiene todos los archivos del proyecto; no solo GameObjects, sino:

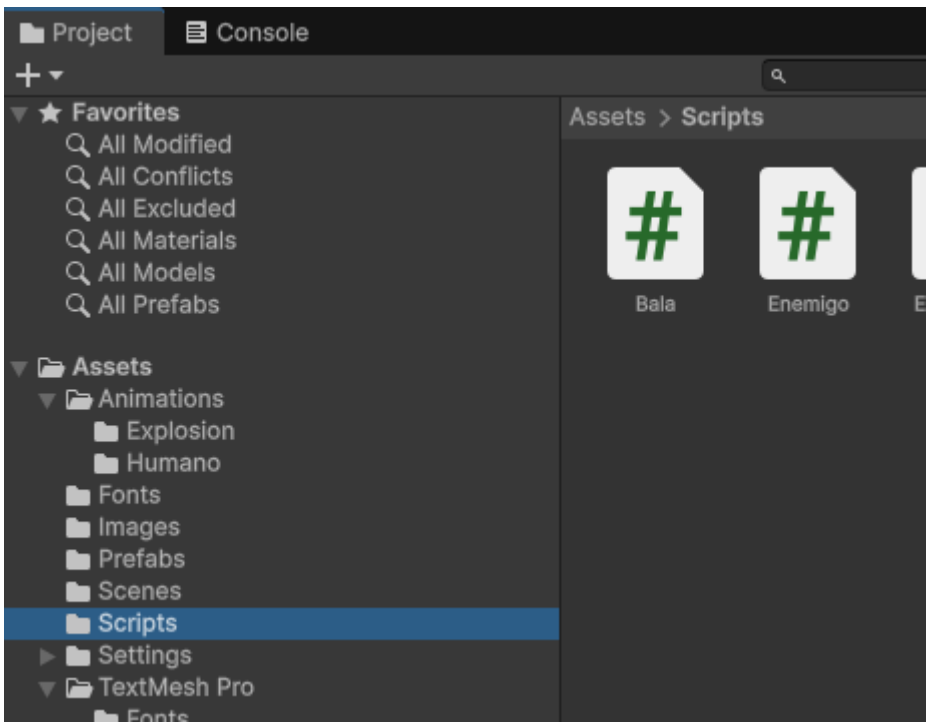
- Scripts (C#)
- Sprites y tiles
- Animaciones

- Prefabs
- Sonidos y música
- Materiales
- Partículas
- Escenas

El orden recomendado es estructurarlo en carpetas como:

- **Scripts**
- **Prefabs**
- **Sprites**
- **Animations**
- **Audio**
- **Scenes**
- **Materials**
- **UI**

Esta organización facilita localizar elementos utilizados por scripts como **Enemigo** (prefabs de bala y explosión), **Humano** (sprite, animaciones, transform de disparo), o **GameManager** (prefab de obstáculos).



### 2.6. Console

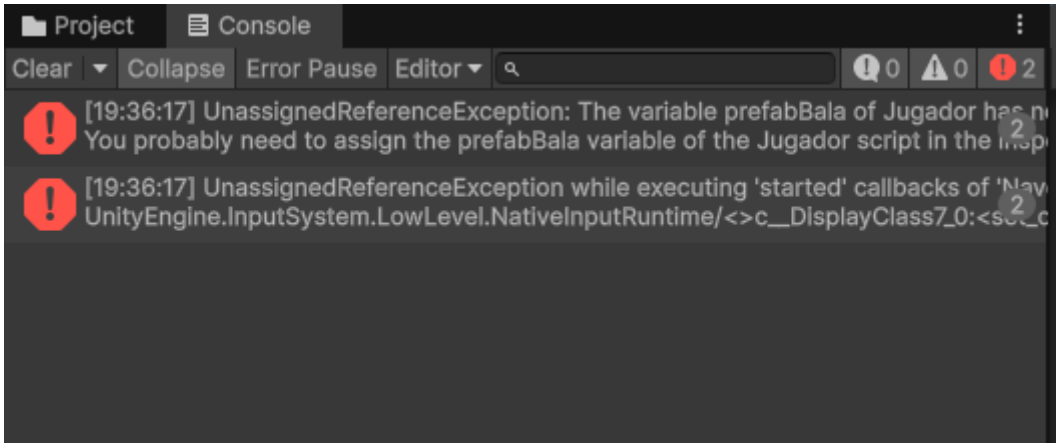
La **Console** muestra:

- Logs (Debug.Log, print)
- Advertencias
- Errores y excepciones
- Información al pulsar Play

Ejemplo presente en varios scripts:

```
print("Direccion: " + direccion);
```

La Console es imprescindible para depurar problemas como valores incorrectos, inputs que no detectan correctamente o referencias no asignadas.




### 2.7. Toolbars y Layouts

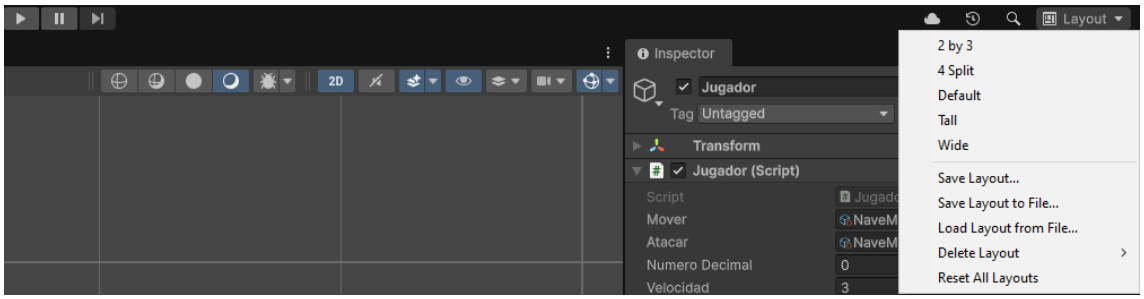
La parte superior del editor contiene la **Toolbar**, con opciones como:

- Botones Play, Pause, Step
- Control de escenas
- Modo de manipulación (Translate, Rotate, Scale…)
- Botón de Layers y Gizmos
- Opciones de diseño y organización del layout

Unity permite guardar diferentes **Layouts** personalizados, por ejemplo:

- Layout para programación
- Layout para animaciones

- Layout para edición de UI
- Layout minimalista para pantallas pequeñas

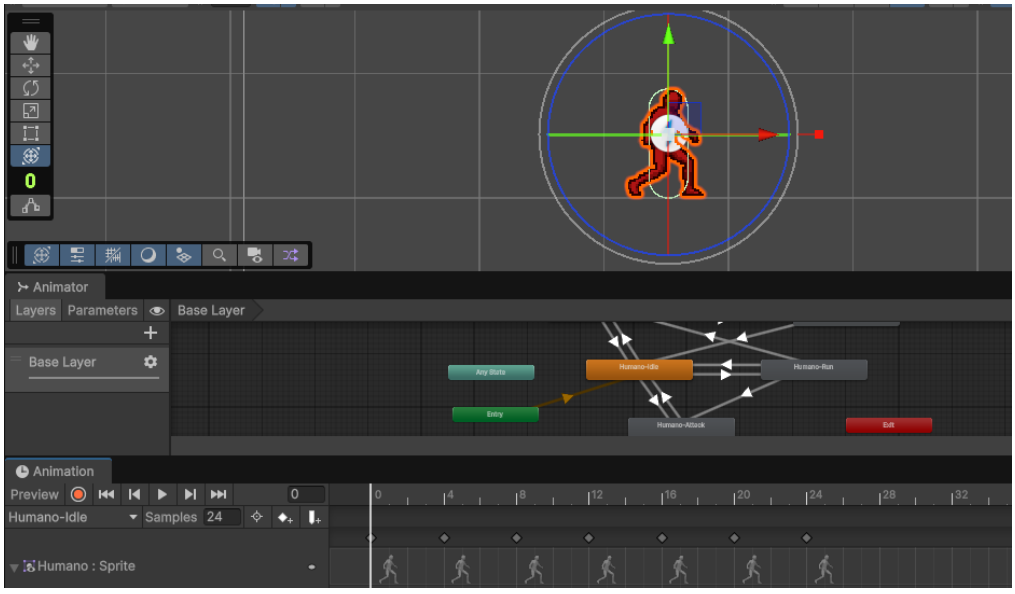



## 2.8. Ventanas adicionales útiles

Unity ofrece muchas ventanas complementarias según la fase del proyecto:

- **Animator** → configuración de estados y transiciones
- **Animation** → edición cuadro a cuadro
- **Sprite Editor** → slicing, pivots, borders
- **Package Manager** → input system, TextMeshPro, Cinemachine…
- **Profiler** → análisis de rendimiento
- **Lighting** → ajustes de iluminación (más relevante en 3D)

Scripts como **Humano** o **Jugador** hacen uso del sistema de animación y del Input System, que se gestionan desde estas ventanas.



### 2.9. Flujo de trabajo: una ventana complementa a otra

El trabajo en Unity suele alternar constantemente entre ventanas. Ejemplos típicos:

- Scene View → colocar objetos
- Inspector → ajustar componentes
- Game View → probar el resultado
- Console → depurar problemas
- Project → localizar assets
- Animator → ajustar transiciones

Este flujo continuo hace que la personalización del espacio de trabajo sea una parte esencial del uso eficiente del editor.